

Relatório — Hello World com TensorFlow Lite Micro no ESP32-S3 (Wokwi)

Aluno: Daniel Quiteque

Disciplina: IA Embarcada e Modelos Compactos — Atividade Avaliativa Prática 4/6

Repositório: <https://github.com/danielquiteque/IA-embarcada/tree/main/atividade-4-hello-world>

1. Objetivo

Reproduzir o exemplo *Hello World* do TFLite Micro: treinar uma rede neural que aproxima $\sin(x)$ no intervalo $[0, 2\pi]$, comprimi-la por quantização int8, convertê-la em um array C e executá-la em um ESP32-S3 simulado no Wokwi, analisando o código e a documentação do exemplo.

2. Passos reproduzidos

Etapas	O que foi feito
Dataset	1000 valores $x \sim U(0, 2\pi)$ e $y = \sin(x)$; 20% para validação
Treino	Keras <code>Dense(32, relu) → Dense(32, relu) → Dense(1)</code> , Adam, MSE, 300 épocas, batch 64
Conversão float	<code>TFLiteConverter.from_keras_model</code> → <code>hello_world_float.tflite</code>
Quantização	Pós-treino <i>full integer</i> a partir do SavedModel, com dataset representativo de 500 amostras, entrada/saída int8 → <code>hello_world_int8.tflite</code>
Array C	<code>.tflite</code> → <code>model.cc</code> (equivalente ao <code>xxd -i</code>) com <code>alignas(8)</code>
Firmware	Exemplo <code>hello_world</code> do esp-tflite-micro 1.4.1, compilado com ESP-IDF para esp32s3
Simulação	Wokwi com <code>diagram.json</code> + <code>wokwi.toml</code> apontando para <code>build/</code>

3. Resultados do modelo

Avaliação em 200 pontos igualmente espaçados em $[0, 2\pi]$:

Modelo	Tamanho	MAE	RMSE
float32	6580 B	0.0124	0.0205
int8	5160 B	0.0191	0.0253
Original do exemplo (16 → 16 → 1, int8)	2488 B	—	—

Parâmetros de quantização do modelo gerado (lidos do flatbuffer):

- entrada: `scale = 0.02463`, `zero_point = -128` → o int8 $[-128, 127]$ cobre $x \in [0, 6.28]$
- saída: `scale = 0.00788`, `zero_point = -2` → cobre $y \approx [-0.99, 1.02]$
- 3 operadores, todos `FULLY_CONNECTED` (o ReLU fica fundido na própria camada)

4. Análise do código

`main.cc` — ponto de entrada `app_main()` (em `extern "C"`, pois o ESP-IDF chama uma função C). Segue o padrão Arduino: `setup()` uma vez e `loop()` a cada 500 ms via `vTaskDelay`, cedendo a CPU ao FreeRTOS

entre inferências.

`main_functions.cc` — a pipeline do TFLite Micro:

1. `tflite::GetModel(g_model)` mapeia o array em memória sem copiar nem fazer parse (flatbuffer), e a versão do schema é conferida.
2. `MicroMutableOpResolver<1>` registra **só** o operador `FullyConnected`. Em MCU não se linka o conjunto completo de operadores: cada op registrado ocupa flash.
3. `MicroInterpreter` recebe um `tensor_arena` estático de 2000 bytes. Não há `malloc`: todos os tensores intermediários são alocados dentro desse buffer por `AllocateTensors()`. Adicionei um `MicroPrintf` com `arena_used_bytes()` para medir o quanto é de fato usado.
4. Em `loop()`, `x` percorre $[0, 2\pi]$ em `kInferencesPerCycle = 20` passos. A entrada é **quantizada** $(x / \text{scale} + \text{zero_point})$, `Invoke()` roda a inferência e a saída é **desquantizada** $((y_q - \text{zero_point}) * \text{scale})$ antes de ser impressa.

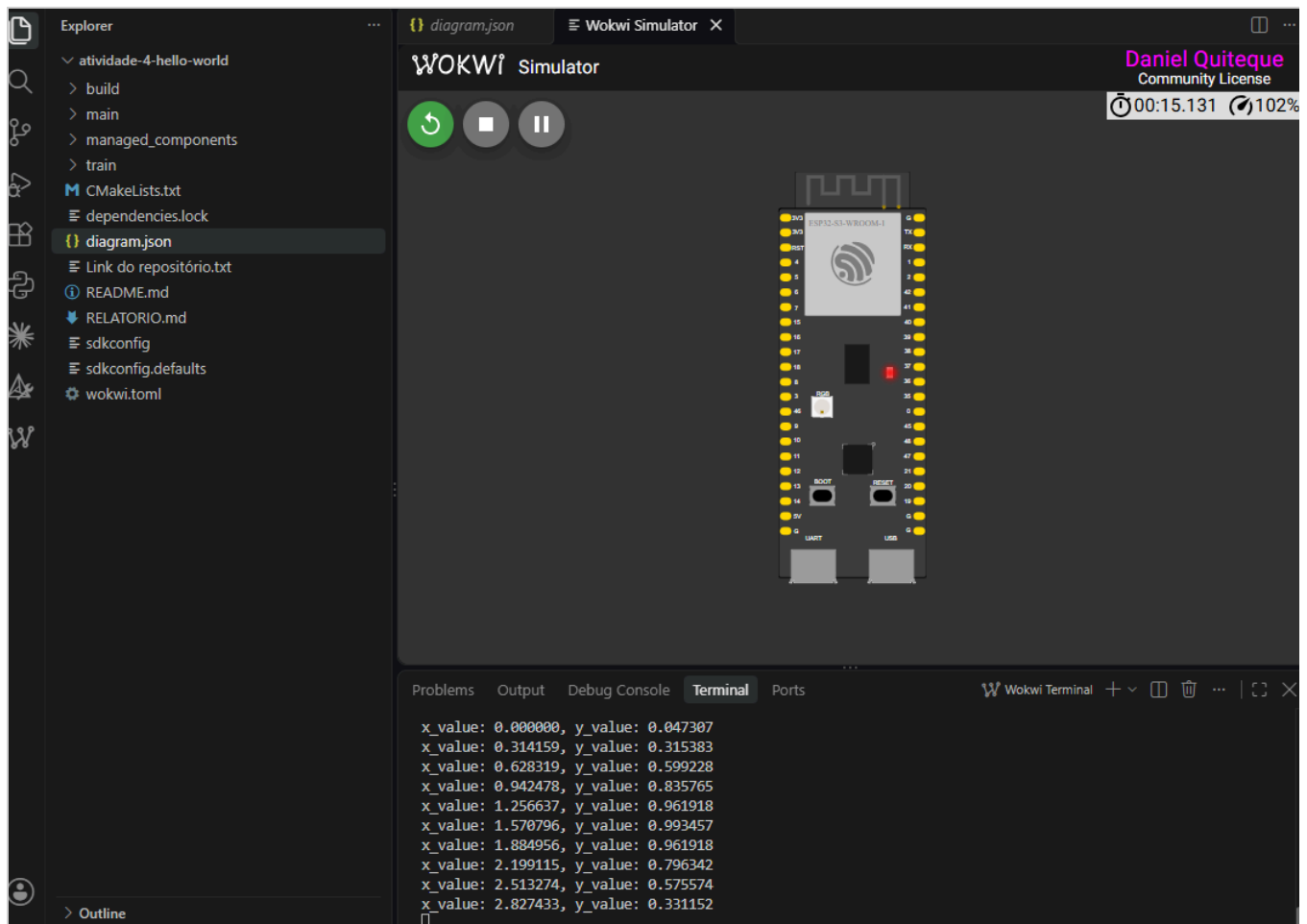
`output_handler.cc` — isola a saída (aqui `MicroPrintf` no serial) para que cada placa possa ter a sua implementação (LED, display, PWM) sem mexer na pipeline.

`model.cc` / `model.h` — o `.tflite` vira um `const unsigned char[]`, que vai para a **flash** (rodata) e não ocupa RAM. Os bytes 4–7 são `TFL3`, o identificador do formato. O `alignas(8)` evita acessos desalinhados ao ler os campos do flatbuffer.

5. Observações

1. **Wokwi e ESP-NN.** O componente compila com `-DESP_NN`, que usa kernels otimizados com instruções SIMD do ESP32-S3 que o Wokwi não simula. Para rodar foi preciso remover essa flag (na aula, comentando a linha no `CMakeLists.txt` do componente; aqui, removendo-a no `CMakeLists.txt` do projeto para não editar código de terceiros). No hardware real, manter o ESP-NN acelera bastante a inferência. Em versões mais novas do `esp-tflite-micro` isso virou a opção `Kconfig` `ESP_TFLITE_MICRO_USE_ESP_NN`.
2. **Quantização reduziu pouco o tamanho total (–22%).** Os pesos ficam 4× menores (`float32` → `int8`), mas o modelo tem só ~1,1 mil parâmetros, então o cabeçalho e os metadados do flatbuffer (nomes de tensores, assinaturas) pesam proporcionalmente muito. Em modelos maiores o ganho tende a ~4×.
3. **Perda de precisão pequena.** O MAE subiu de 0.012 para 0.019, aceitável para a aplicação e em troca de aritmética inteira, que é mais rápida e econômica no MCU.
4. **Treino não determinístico.** Na primeira execução, mesmo com seed fixa, a rede convergiu mal (MAE ≈ 0.29), provavelmente por neurônios ReLU "mortos"; ao repetir, chegou a 0.012. Por isso vale sempre avaliar o `.tflite` final antes de embarcar.
5. **Quantização da entrada sem arredondamento nem saturação.** `int8_t x_q = x / scale + zp` trunca em vez de arredondar e não limita a $[-128, 127]$. Funciona aqui porque `x` nunca sai de $[0, 2\pi]$, mas para dados reais de sensor seria melhor usar `roundf` e saturar o valor.
6. **Tamanho da arena.** Os 2000 bytes são fixados manualmente. Com a impressão de `arena_used_bytes()` que adicionei, a execução no Wokwi mostrou **1260 de 2000 bytes usados (63%)**, sobrando 740 bytes. Na prática, a arena poderia ser reduzida para ~1300–1400 bytes (valor usado mais uma margem), economizando RAM sem risco de falha no `AllocateTensors()`.
7. **Este modelo é maior que o original** (32 neurônios por camada no notebook contra 16 no exemplo), mas ainda cabe com sobra: ~5 KB de flash e 1260 bytes de RAM na arena.

6. Execução no Wokwi

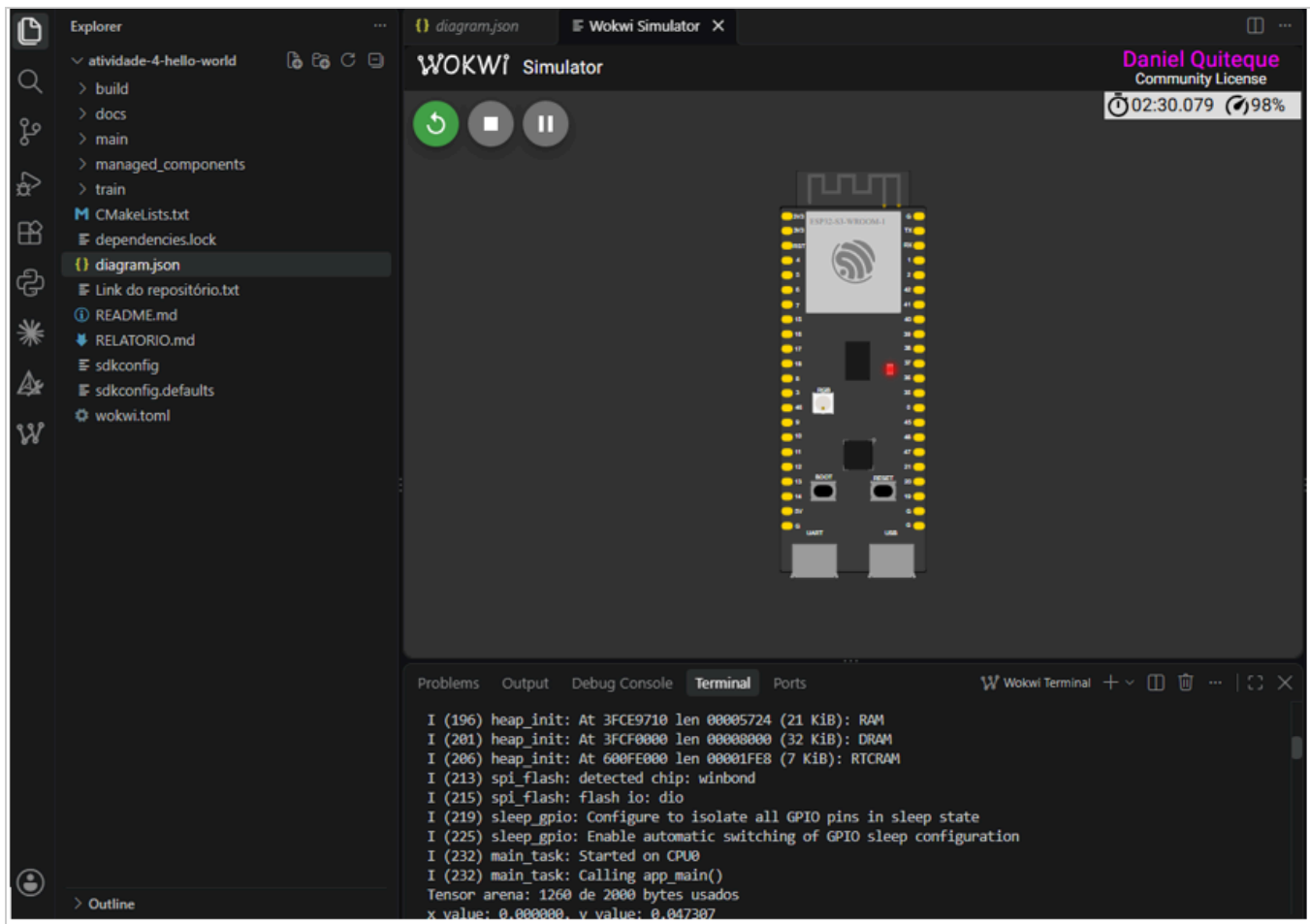


Primeiro meio ciclo impresso pelo ESP32-S3 comparado com o seno exato:

x	y (ESP32, int8)	sin(x)	erro
0.0000	0.0473	0.0000	0.0473
0.3142	0.3154	0.3090	0.0064
0.6283	0.5992	0.5878	0.0114
0.9425	0.8358	0.8090	0.0268
1.2566	0.9619	0.9511	0.0108
1.5708	0.9935	1.0000	0.0065
1.8850	0.9619	0.9511	0.0108
2.1991	0.7963	0.8090	0.0127
2.5133	0.5756	0.5878	0.0122
2.8274	0.3312	0.3090	0.0222

O MAE desses pontos no dispositivo é **0.0167**, coerente com o MAE de 0.0191 do modelo int8 medido no PC. Isso confirma que a quantização da entrada, a inferência e a desquantização da saída no firmware reproduzem o comportamento do modelo convertido. O maior erro ocorre em $x = 0$, na borda do intervalo de treino, onde a rede tem menos amostras ao redor do ponto.

Início da execução, com o log de boot do ESP32-S3 e o uso medido da arena (1260 de 2000 bytes):



7. Extra — nova aplicação: classificação de orientação com MPU6050

Código completo em [atividade-4-extra-orientacao](#).

O Hello World foi transformado em uma aplicação com **sensor novo, dataset novo e modelo retreinado**:

	Hello World	Extra
Sensor	nenhum (x calculado no código)	MPU6050 (acelerômetro + giroscópio) via I2C
Dataset	1000 pontos de $\sin(x)$	8500 leituras rotuladas de acelerômetro e giroscópio
Tarefa	regressão ($1 \rightarrow 1$)	classificação (6 entradas $\rightarrow$ 7 classes)
Modelo	Dense $32 \rightarrow 32 \rightarrow 1$	Dense $16 \rightarrow 16 \rightarrow 7$ + softmax, treinado do zero
Operadores	FullyConnected	FullyConnected + Softmax

Classes: plana (face para cima/baixo), em pé (eixo X para cima/baixo), de lado (eixo Y para cima/baixo) e **em rotação**. A última exige que a rede combine os dois sensores: girando, a orientação deixa de importar.

Dataset. Sem placa física para coletar dados, o dataset foi gerado a partir da física do sensor. Parado, o acelerômetro mede só a gravidade (~ 1 g apontando para cima no referencial da placa), e por isso foram sorteadas direções uniformes na esfera, com magnitude de 0,7 a 1,5 g, ruído de 0,03 g e tremor de até 30 °/s. Em rotação, foram usados de 60 a 250 °/s em um eixo aleatório. Os valores são saturados em ± 2 g e ± 250 °/s, como no driver. O rótulo estático é o eixo com a maior componente da gravidade. O CSV gerado faz parte do repositório.

Resultados (20% de teste):

Modelo	Tamanho	Acurácia
float32	4204 B	97,29%
int8 (embarcado)	4072 B	96,71%

Os erros se concentram perto de 45°, onde a placa está entre duas orientações.

Pipeline no ESP32-S3: leitura I2C (driver da atividade 2/6) → normalização (giroscópio ÷ 250, igual ao treino) → quantização int8 **com lroundf e saturação**, corrigindo a observação 5 → `Invoke()` cronometrado com `esp_timer_get_time()` → `argmax` e desquantização da confiança.

Execução no Wokwi. Com a mesma aceleração (X = 1 g), a classe muda apenas pelo giroscópio:

Aceleração [g]	Rotação [°/s]	Classe prevista	Confiança	Tempo de inferência
1, 0, 0	0, 0, 121	em rotação	100%	1715 µs
1, 0, 0	0, 0, 1	em pé, eixo X para cima	100%	1879 µs

WOKWI Simulator

🔄

■

⏸

MPU6050 Accelerometer + Gyroscope

ACCELERATION

X: 1 g

Y: 0 g

Z: 0 g

ROTATION

X: 0°/sec

Y: 0°/sec

Z: 121°/sec

TEMPERATURE

24.0°C

0

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

UART

USB

Wokwi Terminal

+

▢

🗑

⋮

🔍

✕

I (134774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 116.0 -> em rotacao (100%) [1715 us]

I (135274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 116.0 -> em rotacao (100%) [1715 us]

I (135774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 116.0 -> em rotacao (100%) [1715 us]

I (136274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (136774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (137274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (137774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (138274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (138774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

I (139274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.0 0.0 121.0 -> em rotacao (100%) [1715 us]

WOKWI Simulator

Daniel Quiteque
Community License

17:04.667 100%

MPU6050 Accelerometer + Gyroscope

ACCELERATION

X: 1 g

Y: 0 g

Z: 0 g

ROTATION

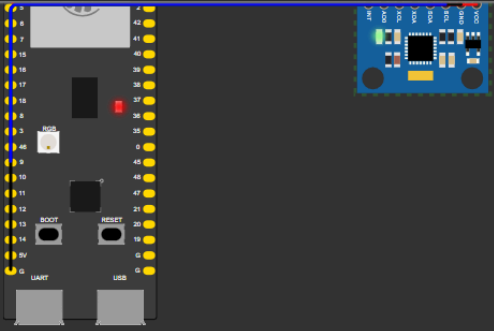
X: 0°/sec

Y: 0°/sec

Z: 1°/sec

TEMPERATURE

24.0°C



Problems Output Debug Console Terminal Ports

Wokwi Terminal

```
I (1019774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1020274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1020774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1021274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1021774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1022274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1022774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1023274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1023774) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
I (1024274) orientacao: Acel[g] 1.00 0.00 0.00 | Giro[dps] 0.00 0.00 1.0 -> em pe, eixo X para cima (100%) [1879 us]
```

Os ~1,8 ms por inferência usam os kernels de referência, já que o ESP-NN foi desativado para o Wokwi; em hardware real, com o ESP-NN, o tempo tende a cair.

Limitações: o dataset é sintético. Com uma placa real, o próximo passo seria coletar leituras pelo serial (como na atividade 2/6) e fazer *fine-tuning*. Além disso, uma única leitura não separa "girando devagar" de "parado com tremor", o que uma janela de leituras resolveria (como no UCI HAR).